
List of Figures

1	Horloge scalaire	7
2	Horloge vectorielle	9
3	Horloge matricielle	11
4	Horloges scalaires simulées en C.	13
5	Horloges vectorielles simulées en C.	14
6	Horloges matricielles simulées en C.	15
7	Horloges scalaires simulées en Java.	16
8	Horloges vectorielles simulées en Java.	16
9	Horloges matricielles simulées en Java.	17

Mini Projet:Algorithmiques Réparties

I. Introduction

Dans les systèmes distribués, la gestion de l'ordre des événements est essentielle, car l'absence d'une horloge globale rend complexe la synchronisation entre processus. Pour résoudre ce problème, on utilise des horloges logiques : **horloge scalaire**, **horloge vectorielle** et **horloge matricielle**.

Ce projet vise à simuler le fonctionnement de ces trois types d'horloges en utilisant les langages **C** et **Java**, avec une communication entre processus via **sockets TCP**. Il permet de visualiser la progression des horloges lors d'événements locaux, d'envois et de réceptions de messages.

II. Objectifs du projet

- Implémenter des processus communicants dans un environnement distribué simulé.
- Utiliser les sockets TCP pour permettre la communication entre processus.
- Mettre en œuvre et comparer les trois types d'horloges logiques.
- Vérifier les relations de causalité entre événements.
- Étudier le comportement des horloges en cas de réception de messages hors ordre.

III. Architecture de l'application

1. Composants

- **Serveur** : Attend la connexion des clients et orchestre le déroulement des itérations.

-
- **Clients (P0 à Pn)** : Simulent les processus distribués. Chacun maintient sa propre horloge et communique avec les autres selon un scénario prédéfini.

2. Communication

Les processus communiquent entre eux via des **sockets TCP**. Chaque message envoyé contient :

- Un identifiant de l'événement.
- L'horloge actuelle du processus émetteur.

3. Scénario global

Chaque client :

- Réalise des événements locaux.
- Envoie ou reçoit des messages.
- Met à jour son horloge selon le type utilisé.

4. Scénario de fonctionnement

Fonctionnement global : Le serveur attend que tous les clients se connectent avant de commencer le traitement des itérations.

Les clients effectuent leurs opérations locales, puis, selon le scénario, envoient des messages à d'autres clients et attendent des messages en retour.

La communication se fait en utilisant des horloges distribuées pour gérer les événements dans le système distribué, garantissant ainsi une synchronisation entre les clients.

Après chaque itération, le serveur envoie un signal "NEXT" pour indiquer aux clients de passer à l'itération suivante.

3. Étapes de fonctionnement

Étape 1 : Connexion des clients au serveur

- **Serveur** : Attend que les 4 clients se connectent.
- **Clients** : Chacun se connecte au serveur sur le port spécifié.

-
- Chaque client initialise son horloge distribuée (scalaire, vectorielle, ou matricielle).

Étape 2 : Itération 1

- **P0** : Exécute une instruction locale, envoie à P1, puis attend jusqu'à ce qu'il reçoive de P3.
- **P1** : Exécute une instruction locale, reçoit de P0, puis envoie à P2.
- **P2** : Exécute une instruction locale, reçoit de P1, puis envoie à P3.
- **P3** : Exécute une instruction locale, reçoit de P2, puis envoie à P0.

Étape 3 : Itération 2

- **P0** : Exécute une instruction locale, envoie à P2, puis attend jusqu'à ce qu'il reçoive de P2.
- **P1** : Exécute une instruction locale, envoie à P3, puis attend jusqu'à ce qu'il reçoive de P3.
- **P2** : Exécute une instruction locale, reçoit de P0, puis envoie à P0.
- **P3** : Exécute une instruction locale, reçoit de P1, puis envoie à P1.

Étape 4 : Itération 3

- **P0** : Exécute une instruction locale, envoie à P3, puis attend jusqu'à ce qu'il reçoive de P1.
- **P1** : Exécute une instruction locale, envoie à P0.
- **P2** : Exécute une instruction locale, envoie à P1.
- **P3** : Exécute une instruction locale, reçoit de P0, puis envoie à P1.

Étape 5 : Itération 4

- **P0** : Exécute une instruction locale, envoie à P1, puis attend jusqu'à ce qu'il reçoive de P3.
- **P1** : Exécute une instruction locale, reçoit de P0, puis envoie à P2.
- **P2** : Exécute une instruction locale, reçoit de P1, puis envoie à P3.
- **P3** : Exécute une instruction locale, reçoit de P2, puis envoie à P0.

Étape 6 : Itération 5

Chaque client exécute une instruction locale.

4. Communication et synchronisation

Les messages entre les clients sont envoyés selon l'ordre spécifié dans le scénario de chaque itération. Chaque message contient l'horloge du client émetteur et un identifiant d'événement.

Après chaque itération, le serveur envoie un signal "NEXT" pour que les clients passent à l'itération suivante. Chaque client attend de recevoir ce signal avant de commencer une nouvelle itération.

La synchronisation entre les processus est gérée à l'aide des horloges logiques distribuées, ce qui permet de garantir la cohérence des événements dans le système distribué.

5. Prérequis et Guide d'exécution

Prérequis

En langage C :

- **Compilateur** : MinGW (Minimalist GNU for Windows)
- **Bibliothèque réseau** : WinSock2 (native à Windows, incluse via `#include <winsock2.h>`)
- **Éditeur** : Visual Studio Code
- **Terminal** : PowerShell ou l'invite de commandes

En langage Java :

- **Kit de développement Java (JDK)** : Version 8
- **Éditeur** : Visual Studio Code
- **Interface** : Ligne de commande

Guide d'exécution

En langage C

Compilation :

- Serveur : `gcc server.c -o server -lws2_32`
- Client : `gcc client.c -o client -lws2_32`

Exécution :

-
1. Lancer le serveur : `./server`
 2. Lancer le script PowerShell : `./execute.ps1`

Remarque : Le script `execute.ps1` démarre automatiquement 4 clients avec le même type d'horloge logique. Le type d'horloge est à spécifier dans le script :

- 0 → Scalaire
- 1 → Vectorielle
- 2 → Matricielle

En langage Java

Compilation :

- `javac *.java`

Exécution :

1. Lancer le serveur :
 - Scalaire : `java Server`
 - Vectorielle : `java ServerV`
 - Matricielle : `java ServerM`
2. Lancer les clients dans 4 terminaux :
 - Scalaire : `java Client <id>`
 - Vectorielle : `java ClientV <id>`
 - Matricielle : `java ClientM <id>`

IV. Horloges logiques implémentées

A. Horloge Scalaire (Lamport)

- Chaque processus maintient un entier L_i .
- Lors d'un événement local : $L_i = L_i + 1$
- Lors de l'envoi d'un message : $L_i = L_i + 1$, message contient L_i
- À la réception : $L_i = \max(L_i, L_m) + 1$

Introduction

L'algorithme des horloges scalaires, introduit par Leslie Lamport, permet de maintenir un ordre logique partiel entre les événements d'un système distribué. Chaque processus maintient un compteur qui s'incrémente à chaque événement, et lors de l'envoi/réception d'un message.

Diagramme illustratif

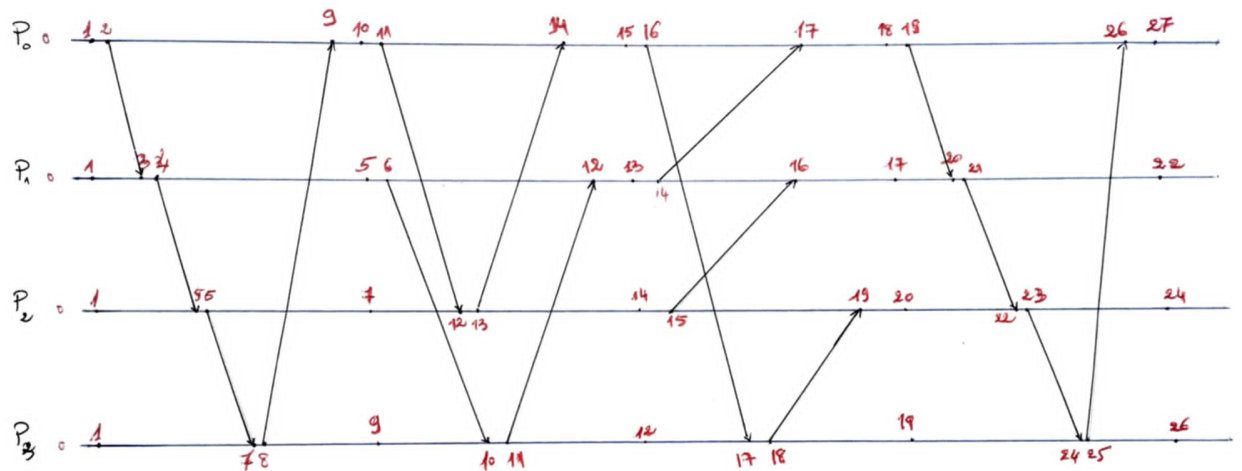


Figure 1: Horloge scalaire

Légende du diagramme

- **Processus P0 à P3 :** Représentés par des lignes horizontales.
- **Événements :** Numérotés en rouge, ils symbolisent des actions locales, des envois ou des réceptions de messages.
- **Flèches :** Indiquent les messages échangés entre processus (envoi →réception).
- **Horloges scalaires :** En noir au-dessus de chaque événement, elles indiquent le temps logique selon Lamport.

Explication du fonctionnement

1. Incrémentation locale

Lorsqu'un processus effectue une action locale (sans communication), il incrémente simplement son horloge de 1.

2. Envoi de message

Avant l'envoi d'un message, le processus incrémente son horloge, et le message contient cette valeur.

3. Réception de message

Lorsqu'un processus reçoit un message, il met à jour son horloge à :

$$\text{Horloge locale} = \max(\text{Horloge locale}, \text{Horloge message}) + 1$$

4. Ordre partiel

La relation de causalité est respectée : si un événement A envoie un message à un événement B, alors A précède causalement B.

Conclusion

Ce mécanisme assure une cohérence temporelle dans les systèmes distribués sans besoin d'horloge physique. L'horloge scalaire est simple mais ne permet pas de capturer toute la causalité (contrairement aux horloges vectorielles ou matricielles).

B. Horloge Vectorielle

- Chaque processus maintient un vecteur $V_i[0..n-1]$.
- Événement local : $V_i[i] = V_i[i] + 1$
- Envoi : $V_i[i] = V_i[i] + 1$, message contient V_i
- Réception de V_j : $V_i[k] = \max(V_i[k], V_j[k])$ pour tout k , puis $V_i[i] = V_i[i] + 1$

Introduction

Les horloges vectorielles permettent de détecter toutes les relations de causalité entre événements dans un système distribué. Chaque processus conserve un vecteur représentant l'évolution de l'ensemble des horloges.

Diagramme illustratif

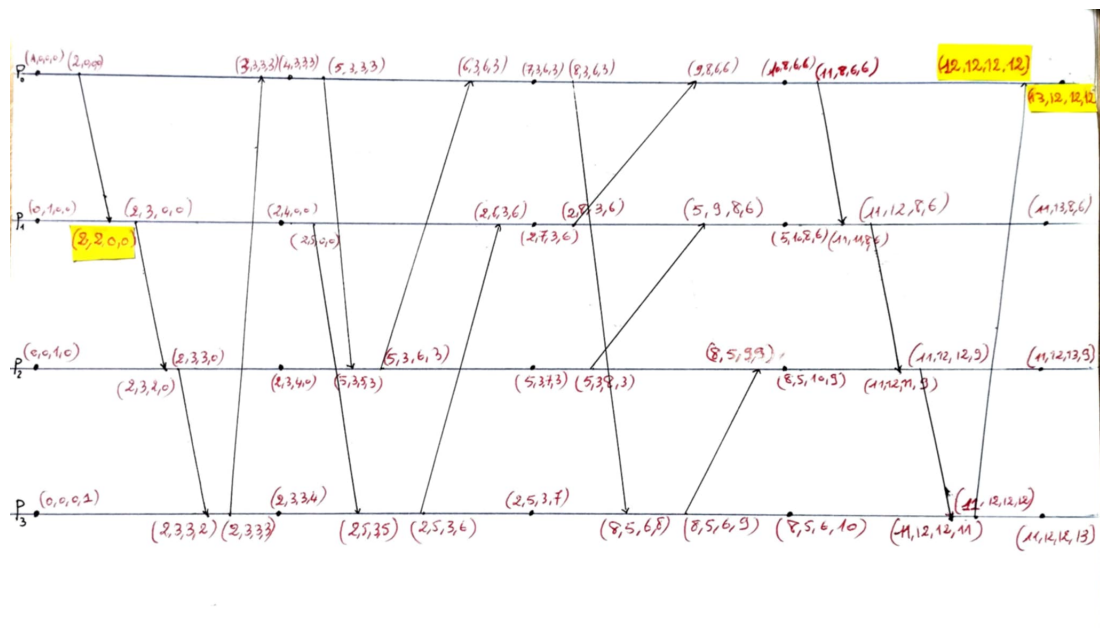


Figure 2: Horloge vectorielle

Légende du diagramme

- Processus P_0 à P_3 : Lignes horizontales.
- Événements : Numérotés en rouge.
- Messages : Flèches entre les processus.
- Vecteurs d'horloge : Affichés au-dessus de chaque événement.

Explication du fonctionnement

1. Incrémentation locale

Chaque processus incrémente la composante correspondant à son propre index.

2. Envoi de message

Avant d'envoyer un message, le processus incrémente sa propre composante, et envoie une copie de son vecteur.

3. Réception de message

Le processus récepteur compare son vecteur avec celui reçu, fait le maximum pour chaque composante, puis incrémente sa propre composante.

4. Relation de causalité

Si $V_a < V_b$ (composante par composante), alors l'événement a précède b .

Conclusion

L'horloge vectorielle permet de capter toute la causalité dans un système distribué, mais au prix d'une complexité mémoire de $O(n)$ par processus.

C. Horloge Matricielle

- Chaque processus maintient une matrice $HM_i[n][n]$
- Événement local : $HM_i[i][i] = HM_i[i][i] + 1$
- Envoi de P_i à P_j :
 - $HM_i[i][i] = HM_i[i][i] + 1$
 - $HM_i[i][j] = HM_i[i][j] + 1$
 - Message contient une copie Em de la matrice
- Réception de Em par P_j , si $Em[k][j] \leq HM_j[k][j]$ pour tout $k \neq i, j$:
 - $HM_j[j][i] = HM_j[j][i] + 1$

- $HM_j[j][j] = HM_j[j][j] + 1$
- Pour tout $k \neq i, j, l \neq i$: $HM_j[k][l] = \max(HM_j[k][l], Em[k][l])$

Introduction

L'horloge matricielle est une généralisation de l'horloge vectorielle, permettant non seulement de connaître les relations de causalité directes, mais aussi de reconstituer les vues globales de chaque processus sur l'état du système.

Diagramme illustratif

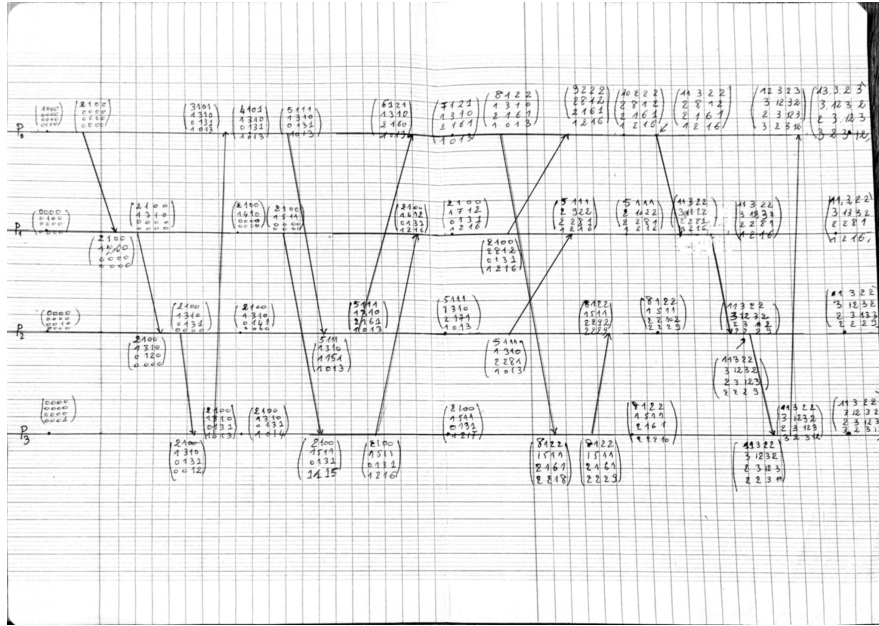


Figure 3: Horloge matricielle

Légende du diagramme

- Processus P0 à P3 : Représentés horizontalement.
- Événements : Numérotés et annotés.

-
- **Matrices d'horloge** : Associées à chaque événement.
 - **Flèches** : Messages échangés entre processus avec conditions de délivrance.

Explication du fonctionnement

1. Incrémentation locale

Le processus incrémente sa propre cellule diagonale $HM[i][i]$.

2. Envoi de message

Il incrémente aussi la cellule de communication $HM[i][j]$, en plus de $HM[i][i]$, et envoie la copie.

3. Réception de message

Si la condition de causalité est respectée, le message est délivré, le récepteur met à jour sa ligne et fusionne les matrices.

4. Vue globale

Chaque processus peut reconstruire une estimation globale de l'état logique de tous les processus à tout moment.

Conclusion

Bien que coûteuse en mémoire ($O(n^2)$), l'horloge matricielle offre une précision maximale sur la causalité et la connaissance distribuée dans un système.

V. Défis rencontrés

- **Synchronisation des messages** : éviter les conflits lors de réceptions multiples.
- **Complexité des horloges matricielles** : vérifier les conditions de délivrance.
- **Affichage clair de l'évolution des horloges** : développement de logs détaillés.

VI. Résultats obtenus

A. Résultats en C

- Fonctionnement correct des horloges dans un scénario multi-processus sous Windows avec Winsock2.
- Validation des relations de causalité grâce à l'implémentation des horloges scalaires, vectorielles et matricielles.
- Détection des réceptions précoces avec blocage temporaire des messages invalides dans l'horloge matricielle.

Illustrations des horloges scalaires pour chaque client en C :

```
[Client 0] Iteration 5  
Processus 0: Execution locale...  
Resultat: 190  
Horloge scalaire mise a jour: 27
```

(a) Client 1

```
[Client 1] Iteration 5  
Processus 1: Execution locale...  
Resultat: 190  
Horloge scalaire mise a jour: 22
```

(b) Client 2

```
[Client 2] Iteration 5  
Processus 2: Execution locale...  
Resultat: 190  
Horloge scalaire mise a jour: 24
```

(c) Client 3

```
[Client 3] Iteration 5  
Processus 3: Execution locale...  
Resultat: 190  
Horloge scalaire mise a jour: 26
```

(d) Client 4

Figure 4: Horloges scalaires simulées en C.

Illustrations des horloges vectorielles pour chaque client en C :

```
[Client 0] Iteration 5  
Processus 0: Execution locale...  
Resultat: 190  
Horloge vectorielle mise a jour: [13,12,12,12]
```

(a) Client 1

```
[Client 1] Iteration 5  
Processus 1: Execution locale...  
Resultat: 190  
Horloge vectorielle mise a jour: [11,13,8,6]
```

(b) Client 2

```
[Client 2] Iteration 5  
Processus 2: Execution locale...  
Resultat: 190  
Horloge vectorielle mise a jour: [11,12,13,9]
```

(c) Client 3

```
[Client 3] Iteration 5  
Processus 3: Execution locale...  
Resultat: 190  
Horloge vectorielle mise a jour: [11,12,12,13]
```

(d) Client 4

Figure 5: Horloges vectorielles simulées en C.

Illustrations des horloges matricielles pour chaque client en C :

```
[Client 0] Iteration 5
Processus 0: Execution locale...
Resultat: 190
Horloge matricielle mise a jour:
[13,3,2,3]
[3,12,3,2]
[2,3,12,3]
[3,2,3,12]
```

(a) Client 1

```
[Client 1] Iteration 5
Processus 1: Execution locale...
Resultat: 190
Horloge matricielle mise a jour:
[11,3,2,2]
[3,13,3,2]
[2,2,8,1]
[1,2,1,6]
```

(b) Client 2

```
[Client 2] Iteration 5
Processus 2: Execution locale...
Resultat: 190
Horloge matricielle mise a jour:
[11,3,2,2]
[3,12,3,2]
[2,3,13,3]
[2,2,2,9]
```

(c) Client 3

```
[Client 3] Iteration 5
Processus 3: Execution locale...
Resultat: 190
Horloge matricielle mise a jour:
[11,3,2,2]
[3,12,3,2]
[2,3,12,3]
[3,2,3,13]
```

(d) Client 4

Figure 6: Horloges matricielles simulées en C.

B. Résultats en Java

- Comportement identique simulé via des threads et sockets Java.
- Synchronisation et causalité bien préservées entre les threads clients.
- Fonctionnement fluide des mécanismes de fusion pour chaque type d'horloge.

Illustrations des horloges scalaires pour chaque client en Java :

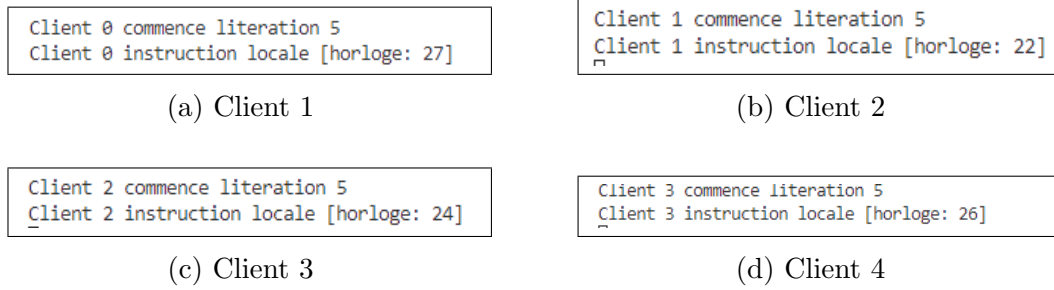


Figure 7: Horloges scalaires simulées en Java.

Illustrations des horloges vectorielles pour chaque client en Java :

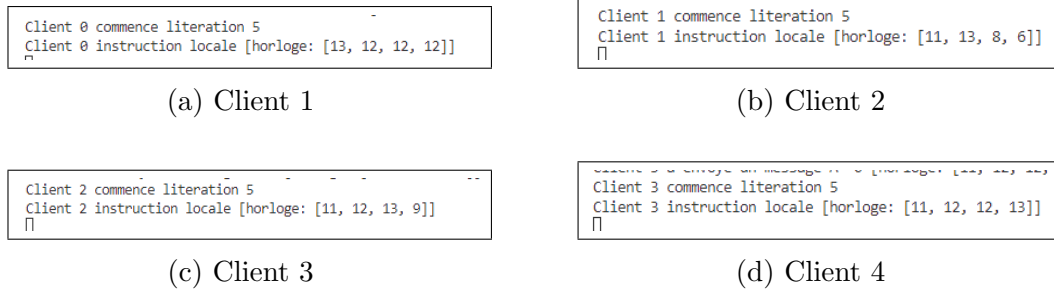


Figure 8: Horloges vectorielles simulées en Java.

Illustrations des horloges matricielles pour chaque client en Java :

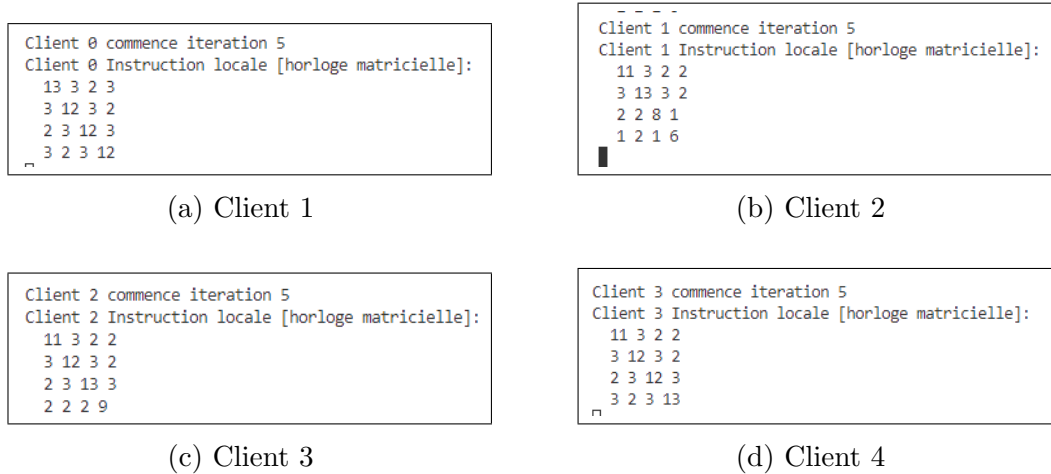


Figure 9: Horloges matricielles simulées en Java.

VII. Conclusion

Ce projet a permis de comprendre en profondeur les mécanismes d’horodatage logique dans les systèmes distribués. Les trois modèles implémentés ont montré leur utilité respective :

- **Scalaire** : simple, mais peu précis pour la causalité.
- **Vectoriel** : très utilisé, efficace pour la causalité.
- **Matriciel** : précis, adapté à la détection de terminaisons.

Ce travail pourrait être enrichi par une visualisation graphique des relations de causalité, ou par l’ajout de pannes pour simuler des conditions de réseau réel.

VIII. Annexes

- Fichier ”README.md” qui décrit le projet en détail
- Fichier ”ExecutionGuide.md” qui décrit comment exécuter l’application en C et en Java
- Fichiers ”Résultats.md” qui contient les résultats complets de l’exécution